PROJECT SPECIFICATION

AI Research & RAG Agent

A production-style, tool-calling research assistant combining agentic workflows, retrieval-augmented generation, and cited report generation.

Prepared for portfolio development · Full-stack AI systems project

Node.js · PostgreSQL + pgvector · Redis · React · Docker

1. Project Overview

We will build an **AI-powered research agent** capable of researching a topic using external sources, retrieving relevant information from its knowledge base, and producing a structured, evidence-backed research report.

Unlike a basic chatbot, the system will be able to **decide which tools to use, gather information, retrieve relevant context, reason over that information, and generate a final response with citations**.

2. Objective

The primary objective is to build a production-style AI system that demonstrates practical understanding of:

LLM APIs | Agentic workflows | Tool calling | RAG | Vector search | Memory | Backend architecture | Async / background processing | Caching | Authentication | API design | Deployment

3. Core Workflow

A typical research request will follow this process:

```
User → Research Agent → Search → Read Sources → Retrieve Knowledge → Reason →
                                Generate Report
```

The agent should be capable of determining when additional research is required instead of blindly generating an answer from the LLM.

4. Major Features

Research

- Accept a research question from the user
- Search the web for relevant information
- Read and extract useful information from sources
- Identify information gaps
- Perform additional searches when necessary
- Generate a structured research report
- Provide source citations

RAG

- Store documents/knowledge in the system
- Generate embeddings

- Store embeddings in PostgreSQL using `pgvector`
- Retrieve relevant information for a query
- Provide retrieved context to the LLM

Agent

The agent will have access to tools such as:

- `searchWeb`
- `readPage`
- `retrieveKnowledge`
- `saveMemory`
- `retrieveMemory`

The agent decides when and how to use these tools.

Memory

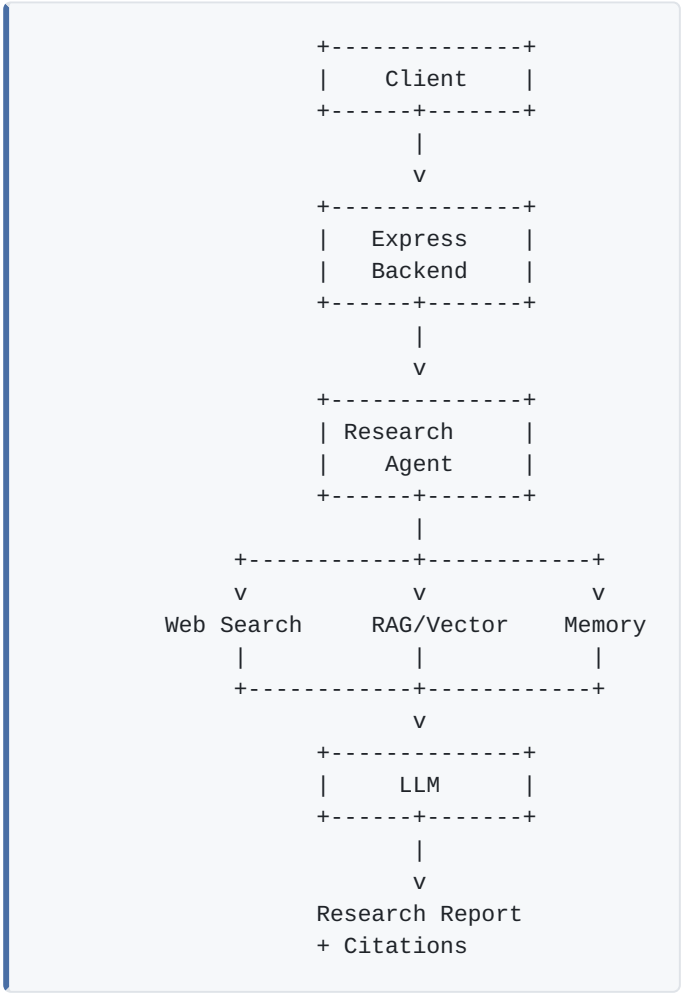
The system will maintain useful information across interactions, allowing the agent to remember relevant facts and previous research context.

Backend

The backend will provide APIs for:

- User authentication
- Research requests
- Research history
- Documents
- Knowledge retrieval
- Memory
- Reports

5. High-Level Architecture



6. Technology Stack

Layer	Technologies
Frontend	React, Tailwind CSS
Backend	Node.js, Express.js
Database	PostgreSQL, pgvector, Redis
AI	LLM API, Embedding model, Tool calling
Infrastructure	Docker, Git/GitHub, Cloud deployment

7. Expected Outcome

> *Receiving a question → researching it → retrieving relevant knowledge → reasoning over multiple sources → producing a structured, cited report.*

The project should also be sufficiently production-oriented that its architecture and implementation can be discussed during a technical interview.

8. Development Approach

The project will be developed incrementally, with each phase introducing a new engineering concept rather than simply adding another screen to the application:

1. **Backend foundation**
2. **LLM integration**
3. **Tool-calling agent**
4. **Web research**
5. **RAG pipeline**
6. **Persistent memory**
7. **Redis and background processing**
8. **Authentication and production APIs**
9. **Docker, testing and deployment**
10. **Frontend and final integration**

9. Success Criteria

The project will be considered complete when:

- The agent can autonomously conduct multi-step research.
- Research is grounded in retrieved information.
- Reports contain source citations.
- Documents can be indexed and retrieved through vector search.
- Agent memory persists between sessions.
- APIs are authenticated and validated.
- Expensive/repeated operations can be cached.
- Long-running tasks can be processed asynchronously.
- The complete application can be deployed.
- The architecture can be clearly explained in a technical interview.